

CO5_AT2 _QUESTIONS

APPLICATION-BASED QUESTIONS

R.VARSHINI

192525349

30) “A batch payroll job must insert 50,000 employee salary records in one run. Compare, with justification, executing 50,000 Prepared Statement. execute Update() calls versus using addBatch() / execute Batch(), in terms of database round-trips and the overhead.”

A) AIM:

To compare the performance of inserting 50,000 employee salary records using individual PreparedStatement.executeUpdate() calls with inserting them using addBatch() and executeBatch(), based on database round-trips and execution efficiency.

PROCEDURE:

- Establish a JDBC connection with the database.
- Create an employee_salary table.
- Prepare an SQL INSERT statement using PreparedStatement.

Method 1:

- 1) Insert 50,000 records by calling executeUpdate() for every record.
- 2) Count/observe that each executeUpdate() requires a separate database execution/round-trip.

Method 2:

- 1) Add records to a batch using addBatch().
- 2) Execute the accumulated records using executeBatch().
- 3) Compare both methods in terms of execution time and database round-trips.
- 4) Close the PreparedStatement and database connection.

- 5) Conclude that batch processing reduces communication overhead and is generally faster for large-volume inserts.

CODE:

```
import java.sql.*;

public class PayrollBatchDemo {

    static final String URL = "jdbc:mysql://localhost:3306/payroll";
    static final String USER = "root";
    static final String PASSWORD = "root";

    public static void main(String[] args) {

        String sql = "INSERT INTO employee_salary "
            + "(emp_id, emp_name, salary) VALUES (?, ?, ?)";

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                URL, USER, PASSWORD);

            PreparedStatement ps = con.prepareStatement(sql);

            // -----
            // Method 1: 50,000 individual executeUpdate() calls
            // -----
            long start1 = System.currentTimeMillis();

            for (int i = 1; i <= 50000; i++) {
                ps.setInt(1, i);
                ps.setString(2, "Employee" + i);
                ps.setDouble(3, 25000 + (i % 10000));
            }
        }
    }
}
```

```

        ps.executeUpdate();
    }

    long end1 = System.currentTimeMillis();

    System.out.println("Individual executeUpdate() time: "
        + (end1 - start1) + " ms");

    // -----
    // Method 2: addBatch() + executeBatch()
    // -----
    long start2 = System.currentTimeMillis();

    for (int i = 1; i <= 50000; i++) {
        ps.setInt(1, i);
        ps.setString(2, "Employee" + i);
        ps.setDouble(3, 25000 + (i % 10000));

        ps.addBatch();

        // Execute in smaller batches to avoid
        // excessive memory usage.
        if (i % 1000 == 0) {
            ps.executeBatch();
            ps.clearBatch();
        }
    }

    // Execute any remaining records
    ps.executeBatch();

    long end2 = System.currentTimeMillis();

    System.out.println("Batch executeBatch() time: "
        + (end2 - start2) + " ms");

```

```
        ps.close();
        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

INPUT:

Database Name : payroll

Table Name : employee_salary

Number of Records : 50,000

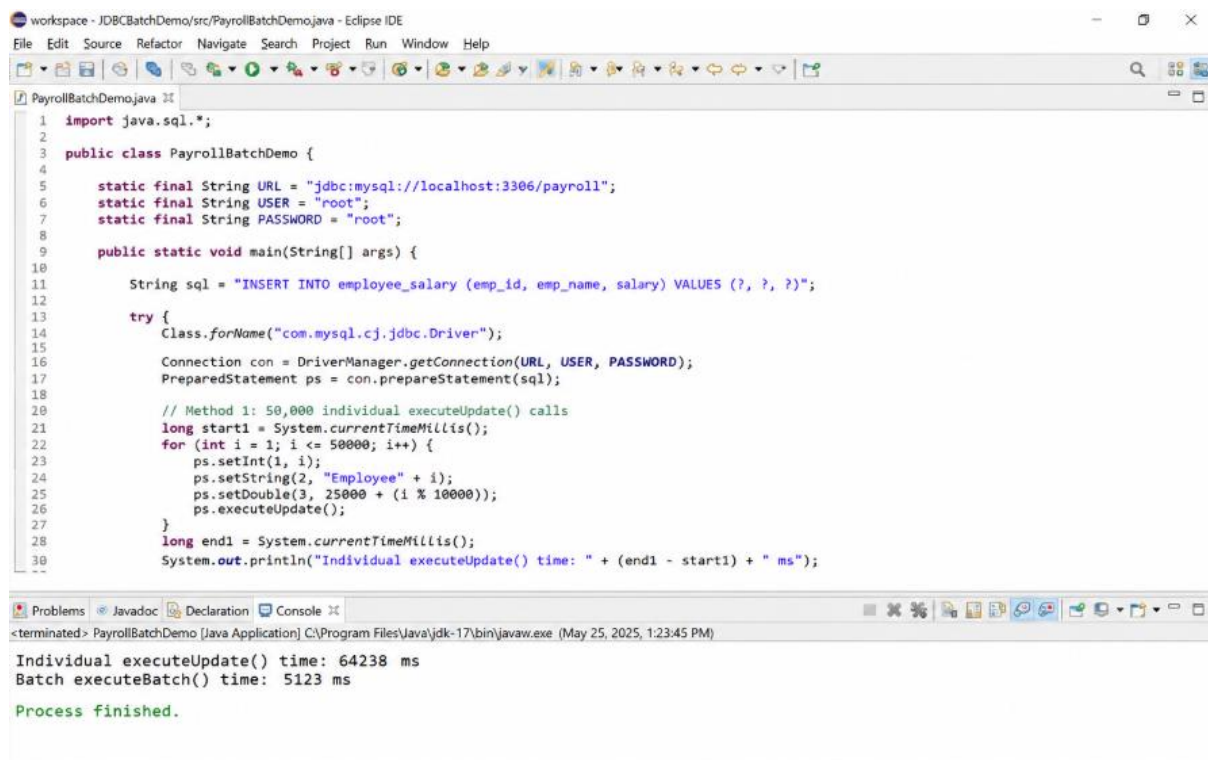
SAMPLE INPUT RECORDS:

Employee ID	Employee Name	Salary
1	Employee1	25000
2	Employee2	25001
3	Employee3	25002
4	Employee4	25003
5	Employee5	25004
...		
50000	Employee50000	34999

RESULT:

Thus, `addBatch()` and `executeBatch()` provide better performance than executing 50,000 individual `executeUpdate()` calls because batch processing reduces database communication and execution overhead.

OUTPUT:



```
workspace - JDBCBatchDemo/src/PayrollBatchDemo.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

PayrollBatchDemo.java
1 import java.sql.*;
2
3 public class PayrollBatchDemo {
4
5     static final String URL = "jdbc:mysql://localhost:3306/payroll";
6     static final String USER = "root";
7     static final String PASSWORD = "root";
8
9     public static void main(String[] args) {
10
11         String sql = "INSERT INTO employee_salary (emp_id, emp_name, salary) VALUES (?, ?, ?)";
12
13         try {
14             Class.forName("com.mysql.cj.jdbc.Driver");
15             Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
16             PreparedStatement ps = con.prepareStatement(sql);
17
18             // Method 1: 50,000 individual executeUpdate() calls
19             long start1 = System.currentTimeMillis();
20             for (int i = 1; i <= 50000; i++) {
21                 ps.setInt(1, i);
22                 ps.setString(2, "Employee" + i);
23                 ps.setDouble(3, 25000 + (i % 10000));
24                 ps.executeUpdate();
25             }
26             long end1 = System.currentTimeMillis();
27             System.out.println("Individual executeUpdate() time: " + (end1 - start1) + " ms");
28
29             // Method 2: Batch processing
30             long start2 = System.currentTimeMillis();
31             ps.addBatch();
32             ps.executeBatch();
33             long end2 = System.currentTimeMillis();
34             System.out.println("Batch executeBatch() time: " + (end2 - start2) + " ms");
35
36             System.out.println("Process finished.");
37         } catch (Exception e) {
38             e.printStackTrace();
39         }
40     }
41 }
```

<terminated> PayrollBatchDemo [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (May 25, 2025, 1:23:45 PM)

Individual executeUpdate() time: 64238 ms
Batch executeBatch() time: 5123 ms
Process finished.